DOCUMENTACIÓN TÉCNICA / 02

Mapa técnico

La última visita a casa

Arquitectura · Componentes · Eventos · Estados

Objetivo: permitir a otra persona localizar rápidamente cada responsabilidad, seguir las conexiones entre scripts y ampliar el prototipo sin conocer previamente el proyecto.

Vista general de la arquitectura

El proyecto utiliza componentes MonoBehaviour en dos escenas de ejecución, contratos mediante interfaces y servicios de coordinación locales a la escena. La interacción física se detecta desde el jugador; la lógica de cada objeto se delega a componentes especializados.

Principios de diseño aplicados

- **Separación de responsabilidades.** El jugador detecta interacciones; los interactuables ejecutan acciones; los managers coordinan estados, puzles e interfaces.
- **Reutilización.** Los contratos IInteractable e IPuzzle y los datos NoteData/ItemData evitan acoplar todas las mecánicas a un objeto concreto.
- **Comunicación híbrida.** Eventos de C# notifican cambios lógicos; UnityEvents enlazan reacciones configurables desde el Inspector.

01 / Estructura de carpetas de Assets/

La carpeta Assets/ contiene todos los recursos, escenas y scripts necesarios para el funcionamiento del videojuego. El contenido desarrollado específicamente para el proyecto se organiza principalmente en carpetas con el prefijo _, (que se añadió para que salgan primero en el proyecto y sean más accesibles) mientras que otras carpetas corresponden a recursos importados y herramientas de Unity.

Carpetas principales

Carpeta	Descripción
_Data/	Datos configurables mediante ScriptableObjects, organizados en Items (objetos, como la llave) y Notes (contenido de las notas).
_Fonts/	Fuentes tipográficas utilizadas en las interfaces y textos del juego, junto con sus recursos de TextMesh Pro.
_Materials/	Materiales de los objetos 3D, organizados por elementos como la casa, la caja, los botones y las notas.
_Models/	Modelos tridimensionales importados en formato FBX, como la llave, la caja, la mecedora y los botones del puzle.
_Prefabs/	Objetos reutilizables de Unity: jugador, interfaz, llave, notas, caja, audio y elementos del escenario.
_Scripts/	Código C# que implementa la lógica del juego, organizado por responsabilidades.
_Shaders/	Shader Graphs personalizados para efectos visuales, como la pixelación y el fundido a negro.
_Sounds/	Efectos de sonido, ambiente, voz y configuración del Audio Mixer.
_Sprites/	Imágenes 2D para notas, símbolos del puzle y elementos de interfaz.
_Textures/	Texturas utilizadas por los materiales 3D, organizadas por elementos del escenario.
_Volumes/	Perfiles de postprocesado e iluminación ambiental de la casa.
Scenes/	Escenas de Unity. Intro es la escena inicial y Casa contiene la experiencia jugable. También se

	conservan escenas anteriores y de pruebas.
Settings/	Configuración de renderizado URP, perfiles gráficos y Build Profiles.
VIdeos/	Vídeos utilizados en la introducción y Render Texture asociada a su reproducción.

Organización interna de _Scripts/

Subcarpeta	Responsabilidad
Managers/	Gestión global del juego, estados, puzles e interfaz (GameManager, PuzzleManager, UIManager).
Player/	Movimiento FPS, detección de interacciones e inventario del jugador.
Interfaces/	Contratos comunes para objetos interactuables y puzles (IInteractable, IPuzzle).
Interactables/	Comportamiento de los objetos interactivos: puertas, llaves, notas y caja.
Puzzles/Box/	Lógica, botones e interfaz del puzle de la caja.
ScriptableObjects/	Definición de las estructuras de datos para objetos y notas.
Animators/	Animaciones programadas, como la rotación de objetos y el balanceo de la mecedora.
Intro/	Control de la secuencia audiovisual introductoria.
Secuencies/	Control de las secuencias de inicio y finalización de la experiencia.
Light/	Control de luces del escenario.

En la raíz de _Scripts/ también se encuentran scripts independientes, como TensionEvent.cs, responsable del evento de tensión, y TransformPositioner.cs, utilizado para posicionar objetos.

Recursos y paquetes importados

Carpeta	Contenido
AK STUDIO ART/	Paquete de recursos 3D de una radio, con su modelo, materiales y prefab.
Keypad/	Paquete externo que contiene modelos, materiales, animaciones, prefabs y scripts de un teclado numérico.
RPG Button 7/	Recursos gráficos para botones de interfaz en diferentes colores y estados.
TextMesh Pro/	Recursos del sistema de texto de Unity, incluidos shaders, fuentes y configuraciones.
TutorialInfo/	Archivos informativos y recursos de ejemplo incluidos con la plantilla del proyecto Unity.

Esta organización permite localizar de forma independiente la lógica de programación, los recursos visuales, los datos configurables y los objetos reutilizables, facilitando el mantenimiento y la ampliación del proyecto.

02 / Componentes y responsabilidades

La siguiente tabla es el punto de entrada para localizar las clases principales. Las rutas son relativas a Assets/_Scripts/.

Control y coordinación

Componente	Responsabilidad Principal
Managers/GameManager	Estado global, control del jugador, comprobación de salida, reinicio y abandono.
Managers/PuzzleManager	Registro por PuzzleId, consulta del conjunto y publicación de PuzzleSolved.

Managers/UIManager	Apertura/cierre de notas, caja, textos, cursor, bloqueo de control... Todo lo que sea interfaz.
Player/FPSController	Movimiento y cámara en primera persona usando CharacterController y PlayerInput.
Player/PlayerInteractor	Raycast desde cámara, para detectar los objetos por sus colliders.
Player/PlayerInventory	Unico objeto portado, recogida, soltado y eventos OnItemPickedUp/OnItemDropped.

Objetos, puzles e interfaces

Componente	Responsabilidad Principal
Interfaces/IInteractable y Interactables/AInteractable	Contrato y clase base de interacción, incluye interact() y prompt cocntextual
Interactables/Notes/NoteInteractable y Interactables/Notes /NoteUI	Lectura de NoteData y Callback al cerrar; presentación visual de la nota
Interactables/BoxInteractable, Puzzles/Box/BoxPuzzle y Puzzles/Box/BoxUI	Entrada a la caja, validación de secuencia y sincronización de botones/slots de UI.
Interactables/ItemInteractable	Recogida de ItemData para objetos más generales.
Interactables/DoorInteractable	Interactable de la puerta con comprobación de llave y progresión para abrirse.
Interfaces/IPuzzle	Contrato común de puzzles
Puzzles/Box/BoxButton	Enum Para clasificar los posibles botones dentro del puzzle

Presentación y secuencias

Componente	Responsabilidad Principal
Intro/IntroSequence	Video de apertura y transición a escena Casa.
Secuencies/StartinSequence y Secuencies/EndingSecuence	Fundido inicial con mensaje de inicio y fundido final y activación de panel final
TensionEvent	Evento de tensión que lanza un evento de Unity y maneja sonido
Light/LightController	Encendo/apagado de luz con sonido
Animators/RotationAnimator	Rotacion animada reutilizable
TransformPositioner	Reposicionamiento entre dos Transforms
RockingChairAnimator	Script para animar la mecedora exclusivamente

03 / Mapa de comunicación

Existen llamadas directas entre componentes, eventos de C# (suscripción/desuscripción en código) y UnityEvents serializados en escenas o prefabs. No hay un bus de eventos global.

Cadena lógica del puzle y desbloqueo de salida

PlayerInteractor interactua con la caja BoxInteractable -> Resuelve Puzzle -> BoxPuzzle: OnSolved -> PuzzleManager: PuzzleSolved -> GameManager: allPuzzleSolved -> DoorInteractable: CanExit() + PlayerInventoty.hasItem(Key)-> EndingSecuence: Cambia estado de game Manager a FINISHED y se activa panel final

Eventos y callbacks de código

Evento	Receptor/efecto
BoxPuzzle.OnSequenceChanged	BoxUI.RefreshSlots actualiza los tres símbolos y el estado del botón Borrar.

BoxPuzzle.OnAttemptFailed	BoxUI.ShowFailure muestra «No era así su rutina».
BoxPuzzle.OnSolved (Action<IPuzzle>)	PuzzleManager.HandlePuzzleSolved recibe el puzle resuelto.
PuzzleManager.PuzzleSolved	GameManager.OnPuzzleSolved actualiza allPuzzlesSolved.
GameManager.OnStateChanged	EndingSequence escucha ENDING e inicia la corrutina de cierre.
UIManager.ShowNote(onClosed)	Al cerrar, ejecuta el callback de NoteInteractable, que invoca su UnityEvent onClosed.
PlayerInventory.OnItemPickedUp / OnItemDropped	Eventos disponibles para otros sistemas; el inventario mantiene CurrentItem.

Conexiones configuradas desde el Inspector

Origen	Acciones observadas
StartingSequence.onFadeCompleted	GameManager.ShowInitialMessage y AudioSource.Play.
NoteInteractable.onClosed	TensionEvent.Trigger. Para la nota de la Cocina
TensionEvent.onTriggered	TransformPositioner.ApplyPosition y RockingChairAnimator.StartRocking. Para movimiento de la silla, El sonido se hace con una corrutina
BoxPuzzle.onSolved	Cierre de BoxUI vía UIManager, apertura visual y desactivación de collider.
BoxPuzzle.onButtonPressed / onCorrect / onFailed	Reproducción de sonidos de pulsación, acierto y fallo.
DoorInteractable.OnDoorOppened / OnDoorClosed	Reproducción de los audios configurados en la puerta.
EndingSequence.onDoorClose / onVoiceSound	DoorInteractable.CloseDoor / AudioSource.Play.

Importante: los UnityEvents dependen de las referencias serializadas en cada instancia. Al crear un nuevo prefab o duplicar un objeto, comprobar estos enlaces en el Inspector.

04 / Estados y progresión del juego

Los estados son un Enum en el GameManager. No se usó patrón State ya que complicaría innecesariamente la lógica para la magnitud de este proyecto, si se ampliase se debería valorar su implementación.

Transiciones implementadas



Figura 1. Transiciones que constituyen el recorrido principal.

Estado	Responsabilidad/Transición
STARTING	Estado inicial de Casa; StartingSequence muestra la entrada, y ShowInitialMessage termina activando ONGOING con un Callback.
ONGOING	Permite movimiento e interacción si playerControlEnabled es true.
ENDING	Se alcanza desde DoorInteractable.OpenDoor tras cumplir requisitos; EndingSequence escucha el cambio.
FINISHED	Se alcanza al terminar el fundido; UIManager muestra el panel final.
PAUSED	Valor previsto en GameState, sin transición específica ni menú de pausa.

Recorrido funcional y condiciones

- **1. Entrada.** IntroSequence reproduce vídeo; al finalizar (o pulsar Espacio) carga Casa. StartingSequence realiza el fundido y llama al mensaje inicial.
- **2. Exploración y notas.** PlayerInteractor detecta objetos mediante raycast; las notas muestran NoteData. Al cerrar la nota configurada en Casa, se dispara TensionEvent una sola vez.
- **3. Evento de tensión.** Se recoloca y balancea la mecedora, se pausa el ambiente, se apagan temporalmente las luces y se reproducen los pasos; el ambiente vuelve con un fade.
- **4. Puzzle de la caja.** Tres pulsaciones (Taza → Radio → Lampara). La solución se verifica al completar la longitud; fallo = mensaje y limpieza; acierto = OnSolved.
- **5. Llave y salida.** ItemInteractable permite llevar una sola llave. DoorInteractable exige HasItem(requiredKey) y GameManager.CanExit() (= ONGOING + todos los puzles resueltos).
- **6. Final.** La puerta cambia el estado a ENDING; EndingSequence reproduce mensaje, voz, cierre y fundido, cambia a FINISHED y muestra el panel.

La lectura de notas orienta al jugador, pero el código no la utiliza como requisito para validar el puzle. El evento de tensión es narrativo; la condición de salida es la llave y la finalización de los puzles registrados.

05 / Escenas, recursos y puntos de extensión

Escenas y prefabs clave

Ruta en Assets/	Uso
Scenes/Intro.unity	Primera escena activa en Build Profiles. Contiene IntroSequence y VideoPlayer.
Scenes/Casa.unity	Escena jugable. Contiene los managers, luces, notas, secuencias y conexiones inspector.
_Prefabs/Character.prefab	FPSController, PlayerInteractor, PlayerInventory y componentes de entrada.
_Prefabs/UI.prefab	Interfaces de nota/caja y elementos de presentación.
_Prefabs/Caja/WoodenCrate.prefab	Caja con BoxPuzzle, BoxInteractable y animación de apertura.
_Prefabs/Enviroment/Door.prefab	Puerta con DoorInteractable y sus eventos.
_Prefabs/Enviroment/Mecedora.prefab	Mecedora con componente visual de balanceo.
_Prefabs/Key.prefab	Ítem recogible asociado a ItemData.

Datos y contratos para reutilizar sistemas

- **NoteData (ScriptableObject).** Almacena contenido y parámetros visuales (fondo, posición, tamaño y tipografía) que NoteUI aplica sin modificar el script.
- **ItemData (ScriptableObject).** Almacena identidad del objeto y posición/rotación de agarre. PlayerInventory compara referencias de ItemData.
- **IInteractable.** Para añadir objetos, implementar o heredar de AInteractable y exponer Interact(PlayerInteractor) y ShowInteractionPrompt().
- **IPuzzle.** Un nuevo puzle debe ofrecer PuzzleId, IsSolved y OnSolved; registrarse en PuzzleManager y emitir el evento cuando se complete.

6 / Criterios de mantenimiento y límites actuales

Decisiones relevantes

Decisión	Motivo y consecuencia
Managers de instancia única	GameManager, UIManager y PuzzleManager exponen Instance para coordinar sistemas en Casa; no se declara persistencia entre escenas.
Eventos lógicos separados de efectos	OnSolved y PuzzleSolved permiten añadir oyentes sin alterar la comprobación del puzle; UnityEvents permiten variar sonidos, animaciones y objetos por escena.
Estado global con enum	GameState concentra la progresión con poca complejidad. No se ha implementado el patrón State con clases por estado.
Inventario de un hueco	Simplifica el prototipo. La llave solo se recoge si no hay otro objeto portado.
Animación visual independiente	RockingChairAnimator mueve rockingVisual y puede dejar fijo el collider en el objeto raíz, evitando colisiones violentas con el jugador.

Puntos a comprobar al ampliar el proyecto

- **Orden de inicialización.** PuzzleManager debe estar presente antes de que BoxPuzzle se registre en Start; GameManager necesita una referencia válida al manager para suscribirse.
- **Enlaces de Inspector.** Los UnityEvents de las escenas y prefabs contienen parte de la lógica. Revisar el destino de cada evento al duplicar objetos.
- **Estados y pausa.** PAUSED está declarado, pero no equivale a un menú ni a una pausa funcional; la UI bloquea movimiento mediante SetPlayerControl(false).
- **Contenido futuro.** Para nuevos puzles, extender IPuzzle y su registro. Para varias llaves u objetos, rediseñar PlayerInventory y la condición de salida.